

Biological Mechanisms in Liquid Neural Networks: Deep Dive

1. Neuronal Dynamics: From Biology to LNNs

1.1 Hodgkin-Huxley Model Foundation

The biological basis of LNNs originates from the Hodgkin-Huxley equations describing action potential generation:

$$C_m \frac{dV}{dt} = -g_{Na}m^3h(V - E_{Na}) - g_Kn^4(V - E_K) - g_L(V - E_L) + I_{ext}$$

Where:

- C_m : Membrane capacitance ($1 \mu\text{F}/\text{cm}^2$)
- V : Membrane potential
- g_{Na}, g_K, g_L : Sodium, potassium, and leak conductances
- m, h, n : Gating variables
- E_{Na}, E_K, E_L : Reversal potentials

Gating Variable Dynamics:

$$\frac{dm}{dt} = \alpha_m(V)(1 - m) - \beta_m(V)m$$

$$\frac{dh}{dt} = \alpha_h(V)(1 - h) - \beta_h(V)h$$

$$\frac{dn}{dt} = \alpha_n(V)(1 - n) - \beta_n(V)n$$

LNN Abstraction:

LNNs simplify this to capture essential dynamics:

$$\tau(V) \frac{dx}{dt} = -x + f(I_{syn})$$

The adaptive time constant $\tau(V)$ captures the voltage-dependent dynamics without explicitly modeling ion channels.

1.2 Synaptic Transmission Mechanisms

Biological Synapses:

Chemical Synaptic Transmission:

1. **Vesicle Release:** Probability follows Poisson statistics $P_{\text{release}} = 1 - e^{-n \cdot p}$ where n is vesicle number, p is release probability
2. **Neurotransmitter Diffusion:** Governed by Fick's law $\frac{\partial c}{\partial t} = D \nabla^2 c$
3. **Receptor Binding:** Michaelis-Menten kinetics $r_{\text{bound}} = \frac{r_{\text{max}} \cdot [\text{NT}]}{K_d + [\text{NT}]}$

LNN Implementation:

```
python

def synaptic_dynamics(pre_voltage, post_voltage, weight, tau_syn=5.0):
    # Vesicle release probability (simplified)
    p_release = tf.nn.sigmoid((pre_voltage - threshold) / temperature)

    # Synaptic conductance
    g_syn = weight * p_release

    # Post-synaptic current (AMPA-like)
    I_syn = g_syn * (E_rev - post_voltage)

    # Synaptic depression/facilitation
    tau_recovery = 100.0 # ms
    depression = tf.exp(-dt / tau_recovery)

    return I_syn * depression
```

1.3 Dendritic Computation

Cable Theory in Dendrites:

The cable equation describes voltage propagation:

$$\lambda^2 \frac{\partial^2 V}{\partial x^2} - \tau \frac{\partial V}{\partial t} - V = 0$$

Where:

- $\lambda = \sqrt{r_m / r_i}$: Space constant
- $\tau = r_m \cdot c_m$: Time constant

Multi-compartment LNN Model:

Multi-compartment LNN Model:

python

```
class DendriticLNN(tf.keras.Model):
    def __init__(self, num_compartments=5):
        super().__init__()
        self.compartments = [
            LTCCell(units=32, tau_min=0.1, tau_max=10.0)
            for _ in range(num_compartments)
        ]

        # Coupling strengths between compartments
        self.coupling = tf.Variable(
            tf.random.normal([num_compartments-1]),
            trainable=True
        )

    def call(self, inputs):
        # Distal to proximal processing
        x = inputs
        states = []

        for i in range(len(self.compartments)):
            x, state = self.compartments[i](x, training=True)
            states.append(x)

            # Lateral coupling
            if i > 0:
                coupling_current = self.coupling[i-1] * (states[i-1] - x)
                x = x + 0.1 * coupling_current

        # Somatic integration
        return tf.reduce_sum(tf.stack(states, axis=0), axis=0)
```

2. Insect Navigation Systems

2.1 Mushroom Body Architecture

The insect mushroom body is a masterpiece of efficient computation:

Anatomical Structure:

- **Projection Neurons (PN):** ~50 in *Drosophila*, ~900 in honeybees
- **Kenyon Cells (KC):** ~2000 in *Drosophila*, ~160,000 in honeybees
- **Output Neurons (MBON):** ~34 types in *Drosophila*

Computational Principles:

1. Expansion Coding:

$$KC_i = f \left(\sum_{j \in \text{random}(7)} w_{ij} \cdot PN_j - \theta_i \right)$$

Only ~5% of KCs active at any time (sparse coding).

2. Winner-Take-All Inhibition: Implemented via APL (anterior paired lateral) neuron:

$$I_{APL} = \alpha \sum_i KC_i$$

$$KC_i^{final} = \max(0, KC_i - I_{APL})$$

Complete Implementation:

```
python

class BiologicalMushroomBody(tf.keras.Model):
    def __init__(self,
        num_pn=50,      # Projection neurons
        num_kc=2000,    # Kenyon cells
        num_mbon=34,    # Output neurons
        connection_prob=0.14, # 7/50 = 14% connectivity
        sparsity=0.05): # 5% active KCs
        super().__init__()

        self.num_pn = num_pn
        self.num_kc = num_kc
        self.num_mbon = num_mbon
        self.sparsity = sparsity

        # Random sparse connectivity PN→KC
        self.build_connectivity(connection_prob)

        # Kenyon cells with adaptive thresholds
        self.kc_layer = AdaptiveKenyonCells(num_kc)

        # APL inhibitory neuron
        self.apl = GlobalInhibition()

        # Mushroom Body Output Neurons
```

```
self.mbon_layer = MBONLayer(num_mbon)
```

```
# Dopaminergic modulation for learning
```

```
self.dopamine_neurons = DopamineModulation()
```

```
def build_connectivity(self, prob):
```

```
    """Create random sparse PN→KC connections"""
```

```
    # Binary connectivity matrix
```

```
    connections = tf.random.uniform([self.num_pn, self.num_kc])
```

```
    self.pn_kc_mask = tf.cast(connections < prob, tf.float32)
```

```
    # Synaptic weights (log-normal distribution as in biology)
```

```
    weights = tf.exp(tf.random.normal([self.num_pn, self.num_kc], 0, 0.5))
```

```
    self.pn_kc_weights = weights * self.pn_kc_mask
```

```
def call(self, sensory_input, reward=None):
```

```
    # 1. Projection neuron encoding
```

```
    pn_activity = tf.nn.relu(sensory_input) # Simple rectification
```

```
    # 2. Expansion to Kenyon cells
```

```
    kc_input = tf.matmul(pn_activity, self.pn_kc_weights)
```

```
    # 3. Adaptive thresholding
```

```
    kc_raw = self.kc_layer(kc_input)
```

```
    # 4. Global inhibition (APL)
```

```
    kc_activity = self.apl(kc_raw, target_sparsity=self.sparsity)
```

```
    # 5. Output computation
```

```
    mbon_activity = self.mbon_layer(kc_activity)
```

```
    # 6. Learning via dopamine modulation
```

```
    if reward is not None:
```

```
        self.update_weights(kc_activity, mbon_activity, reward)
```

```
    return mbon_activity, kc_activity
```

```
def update_weights(self, kc, mbon, reward):
```

```
    """Dopamine-modulated STDP learning"""
```

```
    # Compute dopamine signal
```

```
    dopamine = self.dopamine_neurons(reward)
```

```
    # STDP-like weight update
```

```
    learning_rate = 0.01 * dopamine
```

```
    # Update KC→MBON weights
```

```
    delta_w = learning_rate * tf.matmul(
```

```

        tf.transpose(kc),
        mbon = tf.reduce_mean(mbon) # Centered
    )

    self.mbon_layer.update_weights(delta_w)

```

```

class AdaptiveKenyonCells(tf.keras.layers.Layer):
    def __init__(self, units):
        super().__init__()
        self.units = units

        # Each KC has its own threshold
        self.thresholds = tf.Variable(
            tf.ones([units]) * 2.0,
            trainable=False
        )

        # Homeostatic plasticity parameters
        self.target_rate = 0.05 # 5% target firing rate
        self.adaptation_rate = 0.001

    def call(self, inputs):
        # Apply thresholds
        activity = tf.nn.relu(inputs - self.thresholds)

        # Homeostatic adaptation
        current_rate = tf.reduce_mean(tf.cast(activity > 0, tf.float32))

        # Adjust thresholds to maintain target firing rate
        threshold_delta = self.adaptation_rate * (current_rate - self.target_rate)
        self.thresholds.assign_add(threshold_delta)

        return activity

```

```

class GlobalInhibition(tf.keras.layers.Layer):

    """APL neuron providing global inhibition"""

    def call(self, kc_activity, target_sparsity=0.05):
        batch_size = tf.shape(kc_activity)[0]
        num_kc = tf.shape(kc_activity)[1]

        # Calculate how many KCs should be active
        k = tf.cast(num_kc * target_sparsity, tf.int32)

        # Winner-take-all: keep top k activities

```

```
top_k_vals, top_k_idx = tf.nn.top_k(ke_activity, k)
```

```
# Create sparse tensor
```

```
sparse_kc = tf.scatter_nd(  
    tf.stack([  
        tf.repeat(tf.range(batch_size), k),  
        tf.reshape(top_k_idx, [-1])  
    ], axis=1),  
    tf.reshape(top_k_vals, [-1]),  
    [batch_size, num_kc]  
)
```

```
return sparse_kc
```

2.2 Central Complex: The Insect Compass

The central complex (CX) contains the insect's internal compass and vector memory:

Key Components:

1. **Ellipsoid Body (EB)**: Ring attractor for heading
2. **Protocerebral Bridge (PB)**: Visual input integration
3. **Fan-shaped Body (FB)**: Vector memory
4. **Noduli (NO)**: Motor output

Neural Implementation:

```
python
```

```
class CentralComplex(tf.keras.Model):  
    def __init__(self, num_columns=16, num_layers=8):  
        super().__init__()  
        self.num_columns = num_columns # Anatomical columns  
        self.num_layers = num_layers # FB layers  
  
        # Ellipsoid Body - ring attractor  
  
        self.eb_neurons = self.build_ring_attractor()  
  
        # Protocerebral Bridge  
        self.pb_neurons = tf.keras.layers.Dense(num_columns * 2)  
  
        # Fan-shaped Body - vector memory  
        self.fb_layers = [  
            LTCCell(units=num_columns, tau_min=1.0, tau_max=100.0)  
            for _ in range(num_layers)  
        ]
```

```
# Noduli - output
```

```
self.no_neurons = tf.keras.layers.Dense(2) # Steering commands
```

```
def build_ring_attractor(self):
```

```
    """Build ring attractor network for heading"""
```

```
    class RingAttractor(tf.keras.layers.Layer):
```

```
        def __init__(self, num_neurons=60):
```

```
            super().__init__()
```

```
            self.num_neurons = num_neurons
```

```
            # Build connectivity matrix (Mexican hat)
```

```
            angles = tf.linspace(0.0, 2*np.pi, num_neurons)
```

```
            ang_diff = angles[:, None] - angles[None, :]
```

```
            # Wrap angles to  $[-\pi, \pi]$ 
```

```
            ang_diff = tf.atan2(tf.sin(ang_diff), tf.cos(ang_diff))
```

```
            # Mexican hat connectivity
```

```
            self.W = (tf.exp(-ang_diff**2 / 0.5) -  
                      0.5 * tf.exp(-ang_diff**2 / 2.0))
```

```
        def call(self, inputs, current_state=None):
```

```
            if current_state is None:
```

```
                # Initialize with input
```

```
                state = inputs
```

```
            else:
```

```
                state = current_state
```

```
            # Ring attractor dynamics
```

```
            activation = tf.matmul(state, self.W) + inputs
```

```
            # Normalize to maintain bump
```

```
            state = tf.nn.softmax(activation * 10) # Sharp bump
```

```
            return state
```

```
    return RingAttractor()
```

```
def compute_heading(self, eb_activity):
```

```
    """Extract heading from EB activity bump"""
```

```
    angles = tf.linspace(0.0, 2*np.pi, self.num_columns)
```

```
    # Circular mean
```

```
    cos_sum = tf.reduce_sum(eb_activity * tf.cos(angles))
```

```
    sin_sum = tf.reduce_sum(eb_activity * tf.sin(angles))
```

```

heading = tf.atan2(sin_sum, cos_sum)
return heading

def update_vector_memory(self, heading, speed):
    """Update path integration vector in FB"""
    # Convert heading and speed to vector
    velocity = speed * tf.stack([tf.cos(heading), tf.sin(heading)])

    # Update each FB layer (different time constants)
    fb_outputs = []
    for i, fb_layer in enumerate(self.fb_layers):
        # Each layer integrates at different timescale
        time_scale = 2**i # Exponential timescales
        scaled_velocity = velocity / time_scale

        fb_output, _ = fb_layer(
            tf.expand_dims(scaled_velocity, 0),
            training=True
        )
        fb_outputs.append(fb_output)

    return tf.concat(fb_outputs, axis=-1)

```

2.3 LGMD: Collision Detection System

The Lobula Giant Movement Detector is a marvel of efficient computation:

Biological Circuit:

Photoreceptors → Lamina → Medulla → Lobula → LGMD → DCMD → Motor

Detailed Implementation:

```

python

class BiologicalLGMD(tf.keras.Model):
    """
    Biologically accurate LGMD model including all processing layers
    """

    def __init__(self, image_shape=(128, 128)):
        super().__init__()
        self.image_shape = image_shape

        # Photoreceptor layer (logarithmic response)
        self.photoreceptors = PhotoreceptorLayer()

```

```
# Lamina neurons (lateral inhibition)
```

```
self.lamina = LaminaLayer()
```

```
# Medulla layers (ON/OFF pathways)
```

```
self.medulla_on = MedullaON()
```

```
self.medulla_off = MedullaOFF()
```

```
# Lobula computation
```

```
self.lobula_exc = LobulaExcitatory()
```

```
self.lobula_inh = LobulaInhibitory()
```

```
# LGMD neuron
```

```
self.lgmd = LGMDNeuron()
```

```
# DCMD (descending neuron)
```

```
self.dcmd = DCMDNeuron()
```

```
def call(self, image_sequence):
```

```
    """
```

```
    Process image sequence for collision detection
```

```
    Returns: collision probability and time-to-collision
```

```
    """
```

```
    batch_size, time_steps = image_sequence.shape[:2]
```

```
    collision_probs = []
```

```
    ttc_estimates = []
```

```
    for t in range(1, time_steps):
```

```
        # Photoreceptor adaptation
```

```
        photo_current = self.photoreceptors(image_sequence[:, t])
```

```
        photo_prev = self.photoreceptors(image_sequence[:, t-1])
```

```
        # Temporal difference (motion detection)
```

```
        motion = photo_current - photo_prev
```

```
        # Lamina processing (center-surround)
```

```
        lamina_output = self.lamina(motion)
```

```
        # Medulla ON/OFF channels
```

```
        on_channel = self.medulla_on(tf.nn.relu(lamina_output))
```

```
        off_channel = self.medulla_off(tf.nn.relu(-lamina_output))
```

```
        # Lobula processing
```

```
        excitation = self.lobula_exc(on_channel + off_channel)
```

```
        inhibition = self.lobula_inh(
```

```
            tf.reduce_mean(on_channel + off_channel, axis=[1, 2])
```

```
        )
```

```
# LGMD computation
```

```
lgmd_response = self.lgmd(excitation, inhibition)
```

```
# DCMD thresholding
```

```
spike_rate = self.dcmd(lgmd_response)
```

```
# Collision probability
```

```
collision_prob = self.compute_collision_probability(spike_rate)
```

```
# Time-to-collision estimate
```

```
ttc = self.estimate_ttc(excitation, t)
```

```
collision_probs.append(collision_prob)
```

```
ttc_estimates.append(ttc)
```

```
return tf.stack(collision_probs, axis=1), tf.stack(ttc_estimates, axis=1)
```

```
def compute_collision_probability(self, spike_rate):
```

```
    """Map DCMD spike rate to collision probability"""
```

```
    # Sigmoid mapping based on biological data
```

```
    threshold = 50.0 # Hz
```

```
    slope = 0.1
```

```
    return tf.nn.sigmoid((spike_rate - threshold) * slope)
```

```
def estimate_ttc(self, excitation, t):
```

```
    """Estimate time-to-collision from expansion rate"""
```

```
    # Calculate image expansion rate
```

```
    current_size = tf.reduce_sum(excitation)
```

```
    if t > 0:
```

```
        expansion_rate = (current_size - self.prev_size) / current_size
```

```
        # TTC = size / expansion_rate (tau theorem)
```

```
        ttc = 1.0 / (expansion_rate + 1e-6)
```

```
        ttc = tf.clip_by_value(ttc, 0, 10) # Clip to reasonable range
```

```
    else:
```

```
        ttc = tf.constant(10.0)
```

```
    self.prev_size = current_size
```

```
    return ttc
```

```
class PhotoreceptorLayer(tf.keras.layers.Layer):
```

```
    """Logarithmic response like biological photoreceptors"""
```

```
    def __init__(self, adaptation_rate=0.1):
```

```

def __init__(self, adaptation_rate=0.1):
    super().__init__()
    self.adaptation_rate = adaptation_rate
    self.adapted_background = None

```

```

def call(self, image):
    # Logarithmic compression
    compressed = tf.math.log(image + 1.0)

    # Adaptation to background
    if self.adapted_background is None:
        self.adapted_background = compressed
    else:
        self.adapted_background = (
            (1 - self.adaptation_rate) * self.adapted_background +
            self.adaptation_rate * compressed
        )

    # Return contrast
    return compressed - self.adapted_background

```

```

class LGMDNeuron(tf.keras.layers.Layer):
    """The LGMD neuron with multiplicative inhibition"""

```

```

def __init__(self):
    super().__init__()
    # Feed-forward inhibition delay
    self.inhibition_delay = 3 # ms
    self.inhibition_history = []

def call(self, excitation, inhibition):
    # Store inhibition history
    self.inhibition_history.append(inhibition)
    if len(self.inhibition_history) > self.inhibition_delay:
        self.inhibition_history.pop(0)

    # Use delayed inhibition (feed-forward)

    if len(self.inhibition_history) == self.inhibition_delay:
        delayed_inh = self.inhibition_history[0]
    else:
        delayed_inh = inhibition

    # Multiplicative computation
    lgmd = excitation * tf.exp(-delayed_inh)

```

```

# Threshold linearity
lgmd = tf.nn.relu(lgmd - 0.2) # Threshold

```

```
return lgmd
```

2.4 Grid Cells and Place Cells

Grid Cell Model:

```
python

class GridCellNetwork(tf.keras.Model):
    """
    Grid cells with hexagonal firing patterns
    Based on continuous attractor dynamics
    """

    def __init__(self,
                 num_modules=4,
                 grid_scales=[0.3, 0.5, 0.8, 1.3]): # Different scales
        super().__init__()
        self.num_modules = num_modules
        self.grid_scales = grid_scales

        # Each module has different scale
        self.modules = [
            self.build_grid_module(scale)
            for scale in grid_scales
        ]

    def build_grid_module(self, scale):
        """Build a single grid cell module"""
        class GridModule(tf.keras.layers.Layer):
            def __init__(self, scale, grid_size=40):
                super().__init__()
                self.scale = scale
                self.grid_size = grid_size

                # Create hexagonal connectivity
                self.W = self.create_hex_weights()

                # Path integration weights
                self.W_vel = tf.Variable(
                    tf.random.normal([2, grid_size, grid_size]),
                    trainable=True
                )

            def create_hex_weights(self):
                """Create weights for hexagonal pattern"""
```

```
size = self.grid_size
W = tf.zeros([size, size, size, size])
```

```
# Hexagonal lattice vectors
```

```
v1 = tf.constant([1.0, 0.0])
```

```
v2 = tf.constant([0.5, np.sqrt(3)/2])
```

```
for i in range(size):
```

```
    for j in range(size):
```

```
        for k in range(size):
```

```
            for l in range(size):
```

```
                # Distance on torus
```

```
                di = min(abs(i-k), size - abs(i-k))
```

```
                dj = min(abs(j-l), size - abs(j-l))
```

```
                # Hexagonal distance
```

```
                dist = tf.sqrt(di**2 + dj**2 + di*dj)
```

```
                # Mexican hat profile
```

```
                W[i,j,k,l] = (  
                    tf.exp(-dist**2 / (2*self.scale**2)) -  
                    0.5 * tf.exp(-dist**2 / (8*self.scale**2))  
                )
```

```
return tf.reshape(W, [size*size, size*size])
```

```
def call(self, activity, velocity):
```

```
    # Reshape activity
```

```
    batch_size = tf.shape(activity)[0]
```

```
    act_2d = tf.reshape(activity, [batch_size, -1])
```

```
    # Attractor dynamics
```

```
    interaction = tf.matmul(act_2d, self.W)
```

```
    # Path integration
```

```
    vel_shift = tf.tensordot(velocity, self.W_vel, axes=[[1], [0]])
```

```
    vel_shift = tf.reshape(vel_shift, [batch_size, -1])
```

```
    # Update
```

```
    new_activity = act_2d + 0.1 * (interaction + vel_shift - act_2d)
```

```
    # Normalize
```

```
    new_activity = tf.nn.softmax(new_activity * 10)
```

```
return tf.reshape(new_activity,  
                  [batch_size, self.grid_size, self.grid_size])
```

```

return GridModule(scale)

def call(self, position, velocity):
    """
    Update grid cell activity based on movement
    """
    outputs = []
    for module in self.modules:
        activity = self.get_initial_activity(position, module.scale)
        updated = module(activity, velocity)
        outputs.append(updated)

    return tf.stack(outputs, axis=1)

def get_initial_activity(self, position, scale):
    """Initialize grid pattern at position"""
    # Create grid pattern centered at position
    size = 40
    x = tf.linspace(0.0, 1.0, size)
    y = tf.linspace(0.0, 1.0, size)
    xx, yy = tf.meshgrid(x, y)

    # Hexagonal pattern
    k1 = 2*np.pi / scale
    k2 = 2*np.pi / (scale * np.sqrt(3))

    pattern = (
        tf.cos(k1 * (xx - position[0])) +
        tf.cos(k1 * (0.5*(xx - position[0]) +
            np.sqrt(3)/2 * (yy - position[1]))) +
        tf.cos(k1 * (0.5*(xx - position[0]) -
            np.sqrt(3)/2 * (yy - position[1])))
    ) / 3.0

    # Normalize
    pattern = tf.nn.relu(pattern)
    pattern = pattern / (tf.reduce_max(pattern) + 1e-6)

    return tf.expand_dims(pattern, 0)

```

2.5 Ant Navigation: Path Integration and Visual Homing

```

python

class AntNavigationSystem(tf.keras.Model):
    """
    Complete ant navigation system with path integration and landmarks

```

```

"""

def __init__(self):
    super().__init__()

    # Polarization compass
    self.pol_compass = PolarizationCompass()

    # Optic flow processing
    self.optic_flow = OpticFlowProcessor()

    # Path integrator
    self.path_integrator = PathIntegrator()

    # Visual memory (snapshot matching)
    self.visual_memory = VisualMemory()

    # Pheromone system
    self.pheromone_system = PheromoneProcessor()

def navigate(self, visual_input, polarization, has_food=False):
    """
    Main navigation decision
    """
    # Get compass heading
    heading = self.pol_compass(polarization)

    # Compute optic flow
    flow = self.optic_flow(visual_input)

    # Path integration
    home_vector = self.path_integrator.update(heading, flow)

    if has_food:
        # Return home using vector
        direction = -home_vector # Opposite direction

        # Lay pheromone trail

        self.pheromone_system.deposit(strength=1.0)
    else:
        # Search for food
        # Check visual memories
        match_quality = self.visual_memory.match(visual_input)

        if match_quality > 0.8:
            # Near known food location
            direction = self.visual_memory.get_direction()

```

```

else:
    # Follow pheromones or search
    pheromone_gradient = self.pheromone_system.sense()
    if tf.reduce_max(pheromone_gradient) > 0.1:
        direction = self.follow_pheromone(pheromone_gradient)
    else:
        direction = self.random_search()

return direction

```

```

class PolarizationCompass(tf.keras.layers.Layer):
    """
    POL neurons in ant brain detecting skylight polarization
    """

    def __init__(self, num_pol_neurons=16):
        super().__init__()
        self.num_neurons = num_pol_neurons

        # Preferred polarization angles
        self.preferred_angles = tf.linspace(
            0, np.pi, num_pol_neurons
        )

        # LTC for temporal filtering
        self.temporal_filter = LTCCell(
            units=num_pol_neurons,
            tau_min=10.0, # Slow adaptation
            tau_max=100.0
        )

    def call(self, polarization_pattern):
        """
        Extract heading from polarization pattern
        """

        # Compute response of each POL neuron
        responses = []

        for i in range(self.num_neurons):
            angle = self.preferred_angles[i]

            # Malus's law for polarization
            response = tf.reduce_mean(
                tf.cos(polarization_pattern - angle)**2
            )
            responses.append(response)

```

```

responses = tf.stack(responses)

# Temporal filtering
filtered, _ = self.temporal_filter(
    tf.expand_dims(responses, 0),
    training=False
)

# Population vector decoding
vectors = tf.stack([
    tf.cos(2 * self.preferred_angles),
    tf.sin(2 * self.preferred_angles)
], axis=1)

population_vector = tf.reduce_sum(
    filtered[0, :, None] * vectors,
    axis=0
)

# Extract angle
heading = tf.atan2(
    population_vector[1],
    population_vector[0]
) / 2

return heading

```

3. Integration: Complete Bio-Inspired SLAM

3.1 Unified Architecture

```

python

class BioInspiredLiquidSLAM(tf.keras.Model):
    """
    Complete biologically-inspired SLAM using all mechanisms
    """

    def __init__(self, config):
        super().__init__()

        # Visual processing pathway
        self.retina = RetinaModel()
        self.lgmd = BiologicalLGMD()

        # Spatial representation
        self.grid_cells = GridCellNetwork()

```

```

self.place_cells = PlaceCellNetwork()
self.head_direction_cells = HeadDirectionCells()

# Memory systems
self.mushroom_body = BiologicalMushroomBody()
self.hippocampus = HippocampalCircuit()

# Navigation
self.central_complex = CentralComplex()
self.path_integrator = PathIntegrator()

# Motor control
self.motor_cortex = MotorController()

# Neuromodulation
self.dopamine = DopamineSystem()
self.serotonin = SerotoninSystem()

```

```

def process_frame(self, visual_input, proprioception, reward=None):

```

```

    """

```

```

    Process single frame with all biological mechanisms

```

```

    """

```

```

    # 1. Retinal processing

```

```

    retinal_output = self.retina(visual_input)

```

```

    # 2. Collision detection

```

```

    collision_risk, ttc = self.lgmd(visual_input)

```

```

    # 3. Feature extraction via mushroom body

```

```

    mb_features, kc_activity = self.mushroom_body(
        retinal_output, reward
    )

```

```

    # 4. Spatial cells update

```

```

    position_estimate = proprioception['position']

```

```

    heading = proprioception['heading']

```

```

    grid_activity = self.grid_cells(
        position_estimate,
        proprioception['velocity']
    )

```

```

    place_activity = self.place_cells(
        grid_activity,
        mb_features
    )

```

```
hd_activity = self.head_direction_cells(heading)
```

5. Hippocampal processing

```
memory_state = self.hippocampus(  
    place_activity,  
    grid_activity,  
    mb_features  
)
```

6. Path integration

```
home_vector = self.path_integrator.update(  
    heading,  
    proprioception['speed']  
)
```

7. Central complex decision

```
nav_command = self.central_complex(  
    hd_activity,  
    home_vector,  
    mb_features  
)
```

8. Motor output

```
motor_command = self.motor_cortex(  
    nav_command,  
    collision_risk  
)
```

9. Neuromodulation

```
if reward is not None:  
    dopamine_signal = self.dopamine(reward)  
    self.update_weights_global(dopamine_signal)
```

```
return {  
    'position': position_estimate,  
    'heading': heading,  
    'motor_command': motor_command,  
  
    'collision_risk': collision_risk,  
    'place_code': place_activity,  
    'grid_code': grid_activity,  
    'memory': memory_state  
}
```

```
class HippocampalCircuit(tf.keras.Model):
```

```
    """
```

```

def __init__(self):
    super().__init__()

    # Dentate Gyrus (pattern separation)
    self.dg = tf.keras.layers.Dense(2000, activation='relu')

    # CA3 (auto-association)
    self.ca3 = LTCCell(units=1000, tau_min=10, tau_max=100)

    # CA1 (output)
    self.ca1 = CfCCell(units=500)

    # Entorhinal cortex feedback
    self.ec_feedback = tf.keras.layers.Dense(500)

def call(self, place_cells, grid_cells, sensory_features):
    # Combine inputs
    ec_input = tf.concat([
        place_cells,
        tf.reshape(grid_cells, [tf.shape(grid_cells)[0], -1]),
        sensory_features
    ], axis=-1)

    # Pattern separation in DG
    dg_output = self.dg(ec_input)

    # Auto-association in CA3
    ca3_state = tf.zeros([tf.shape(dg_output)[0], 1000])
    ca3_output, _ = self.ca3(dg_output, [ca3_state])

    # Output via CA1
    ca1_state = tf.zeros([tf.shape(ca3_output)[0], 1000])
    ca1_output, _ = self.ca1(ca3_output, [ca1_state])

    # Feedback to entorhinal cortex
    ec_prediction = self.ec_feedback(ca1_output)

    return {
        'memory_trace': ca3_output,
        'output': ca1_output,
        'prediction': ec_prediction
    }

```

4.1 Spike-Timing Dependent Plasticity (STDP)

python

```
class LiquidSTDP(tf.keras.layers.Layer):  
    """  
    STDP with liquid dynamics for continuous-time learning  
    """  
  
    def __init__(self,  
        tau_plus=20.0, # ms  
        tau_minus=40.0, # ms
```

```

        A_plus=0.01,
        A_minus=0.01):
    super().__init__()
    self.tau_plus = tau_plus
    self.tau_minus = tau_minus
    self.A_plus = A_plus
    self.A_minus = A_minus

    # Eligibility traces
    self.pre_trace = None
    self.post_trace = None

    def update_traces(self, pre_spikes, post_spikes, dt):
        """Update eligibility traces"""
        if self.pre_trace is None:
            self.pre_trace = tf.zeros_like(pre_spikes)
            self.post_trace = tf.zeros_like(post_spikes)

        # Exponential decay
        self.pre_trace = self.pre_trace * tf.exp(-dt/self.tau_plus) + pre_spikes
        self.post_trace = self.post_trace * tf.exp(-dt/self.tau_minus) + post_spikes

    def compute_weight_change(self, pre_spikes, post_spikes):
        """Compute STDP weight change"""
        # LTP: post spike after pre spike
        ltp = self.A_plus * tf.matmul(
            tf.expand_dims(self.pre_trace, 1),
            tf.expand_dims(post_spikes, 0)
        )

        # LTD: pre spike after post spike
        ltd = -self.A_minus * tf.matmul(
            tf.expand_dims(pre_spikes, 1),
            tf.expand_dims(self.post_trace, 0)
        )

        return ltp + ltd

```

4.2 Dopaminergic Modulation

python

```

class DopamineSystem(tf.keras.Model):
    """
    Reward prediction and modulation
    """

    def __init__(self):

```

```

super().__init__()

# Reward prediction (TD learning)
self.value_network = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(1)
])

# Different dopamine neuron types
self.da_neurons = {
    'vta': LTCCell(units=50, tau_min=5, tau_max=50), # Reward
    'snc': LTCCell(units=30, tau_min=2, tau_max=20), # Motor
    'pag': LTCCell(units=20, tau_min=10, tau_max=100), # Aversion
}

def call(self, state, reward=None):
    # Predict value
    predicted_value = self.value_network(state)

    if reward is not None:
        # Compute RPE (reward prediction error)
        rpe = reward - predicted_value

        # Different dopamine responses
        da_signals = {}
        for region, neurons in self.da_neurons.items():
            if region == 'vta':
                # VTA responds to positive RPE
                signal = tf.nn.relu(rpe)
            elif region == 'snc':
                # SNc for motor learning
                signal = tf.abs(rpe) * 0.5
            else: # pag
                # PAG for negative RPE
                signal = tf.nn.relu(-rpe)

            da_state = tf.zeros([tf.shape(state)[0], neurons.units])
            da_output, _ = neurons(
                tf.expand_dims(signal, -1),
                [da_state]
            )
            da_signals[region] = da_output

        return da_signals, rpe

    return None, predicted_value

```

5. Emergent Properties and Behaviors

5.1 Self-Organization

The combination of these biological mechanisms leads to emergent behaviors:

- 1. **Map Formation:** Grid cells + place cells naturally form cognitive maps
- 2. **Loop Closure:** Mushroom body enables robust place recognition
- 3. **Obstacle Avoidance:** LGMD provides reflexive collision avoidance
- 4. **Homing Behavior:** Path integration enables return navigation
- 5. **Exploration:** Dopamine modulation drives curiosity

5.2 Robustness and Adaptation

python

```

class AdaptiveBioSLAM(BioInspiredLiquidSLAM):
    """
    Extended system with adaptation and robustness
    """

    def __init__(self, config):
        super().__init__(config)

        # Homeostatic regulation
        self.homeostasis = HomeostaticController()

        # Circadian rhythm
        self.circadian = CircadianClock()

        # Stress response
        self.stress_system = StressResponse()

    def adapt_to_environment(self, environmental_cues):
        """
        Long-term adaptation mechanisms
        """

        # Circadian modulation
        time_of_day = self.circadian.get_phase()

        # Adjust neural parameters based on time
        if time_of_day < 0.25 or time_of_day > 0.75: # Night
            # Increase sensitivity
            self.retina.sensitivity *= 1.5
            self.lgmd.threshold *= 0.8

        # Homeostatic regulation
        activity_level = self.compute_global_activity()
        if activity_level > self.homeostasis.target_high:
            # Reduce excitability
            self.apply_global_inhibition(0.1)
        elif activity_level < self.homeostasis.target_low:
            # Increase excitability
            self.apply_global_excitation(0.1)

```